

Seminararbeit

Simulation und Visualisierung von Infrastrukturmodellen – Master Geodäsie

A-Frame

Einsatz von AR- / VR-Anwendung mittels A-Frame zur kommunalen Öffentlichkeitsarbeit

von

Katharina Schulz

Matrikelnummer: 18103509

08.03.2025

Eidesstattliche Versicherung über die Selbstständigkeit

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die von mir angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Quellen stammen, habe ich gekennzeichnet. Die Arbeit hat noch bei keiner anderen Prüfung in gleicher oder ähnlicher Form vorgelegen.

Bottrop, 08.03.2025

Ort, Datum



Schulz, Katharina

Inhalt

1.	Einleitung.....	5
1.1	Virtual Reality.....	5
1.2	Augmented Reality.....	6
2.	A-frame.....	7
2.1	Funktionsweise	7
2.2	Beispielhafte Erläuterung.....	9
3.	Einsatz in der kommunalen Öffentlichkeitsarbeit	11
3.1	AR	12
3.2	VR	13
4.	Ausblick.....	14
5.	Quellcode	16
5.1	AR	16
5.2	VR	16
6.	Marker und QR-Code-Abbildungen.....	18
6.1	URL Ar-Anwendung.....	18
6.2	URL VR-Anwendung	18
	Literaturverzeichnis	19

Abbildungsverzeichnis

Abbildung 1: AR-/VR-Kontinuum (MATTHIAS KNOLL, 2022)	6
Abbildung 2: Quellcode zur Darstellung einer Box, mit der Kantenlänge 3 in rot.....	7
Abbildung 3: Darstellung einer einfachen A-Frame-Szene	9
Abbildung 4: Quellcode zur Erstellung einer einfachen VR-Umgebung mit der Darstellung einfacher Geometrien	10
Abbildung 5: Einfache AR-Szene mittels A-Frame, Darstellung eines roten Würfels auf dem vorgegebenen Hiro-Marker	10
Abbildung 6: Darstellung des erzeugten Umgebungsmodells in SketchUp.....	12
Abbildung 7: Darstellung der AR-Anwendung (links) und VR-Anwendung (rechts) im Browser.....	14
Abbildung 8: Quellcode AR-Anwendung	16
Abbildung 9: Quellcode VR-Anwendung	17
Abbildung 10: QR-Code AR (links), QR-Code VR (mitte), Marker AR (rechts)	18

Abkürzungsverzeichnis

API	Application Programming Interfaces
AR	Augmented Reality
CDN	Content Delivery Network
ECS	Entity-Component-System
glTF	Graphics Language Transmission Format
HTML	Hypertext Markup Language
HTTPS	Hypertext Transfer Protocol Secure
LoD	Level of Detail
QR	quick response
VR	Virtual Reality
WebGL	Web Graphics Library
WebXR	Web Extended Reality
z.B.	zum Beispiel

1. Einleitung

Mit dem Fortschritt digitaler Technologien gewinnen Augmented Reality (AR) und Virtual Reality (VR) zunehmend an Bedeutung, hierbei sind die Einsatzmöglichkeiten vielfältig. Der Schwerpunkt dieser Arbeit liegt auf dem Einsatz innerhalb der kommunalen Öffentlichkeitsarbeit, insbesondere im Kontext eines Stadtjubiläums. Eine Variante für die einfache und webbasierte Erstellung solcher Anwendungen ist A-Frame, eine auf HTML basierte Webframework.

Diese Arbeit behandelt die Grundlagen von A-Frame, die Anwendung für AR- und VR-Projekte und die spezifischen Einsatzmöglichkeiten im kommunalen Umfeld. Dabei wird detailliert erläutert, wie A-Frame funktioniert, welche technischen Anforderungen bestehen und welche konkreten Umsetzungsmöglichkeiten sich für kommunale Verwaltungen ergeben. Im Rahmen der Öffentlichkeitsarbeit sind sowohl AR-, als auch VR-Anwendungen besonders interessant, um Informationen und Eindrücke direkt an den Bürger weiter geben zu können. Durch das zunehmende digitale Verständnis und eine intuitive Bedienung hat jede Person die Möglichkeit, die Anwendungen eigenständig zu nutzen.

1.1 Virtual Reality

VR ist eine computererzeugte, dreidimensionale, immersive und multisensorische Simulation der Wirklichkeit. Zusammengesetzt aus einer Kombination mehrerer Technologien, in der der Nutzer in Echtzeit innerhalb eines virtuellen Raums interagieren kann. (LANGER, 2020)

Mit VR-Umgebungen lassen sich virtuelle, also real nichtexistierende, künstliche Umgebungen erschaffen, Objekte können hierbei an der Realität angelehnt oder vollkommen davon losgelöst sein. Mithilfe geeigneter Technologie kann Nutzern das Gefühl gegeben werden, Teil der künstlichen Umgebung zu sein, indem sie sich physisch anwesend fühlen. Innerhalb einer virtuellen Welt kann auch die Interaktion mit anderen Personen erfolgen bzw. die optische und akustische Präsentation verschiedener Inhalte. Unterschieden wird zwischen kleinen bzw. 360°-Displays und Head-Mounted-Displays bzw. Head-Up-Displays, auch als VR-Brillen bezeichnet. Des Weiteren können zusätzliche Steuergeräte bzw. mit Sensorik ausgestattete Accessoires genutzt werden. (MATTHIAS KNOLL, 2022)

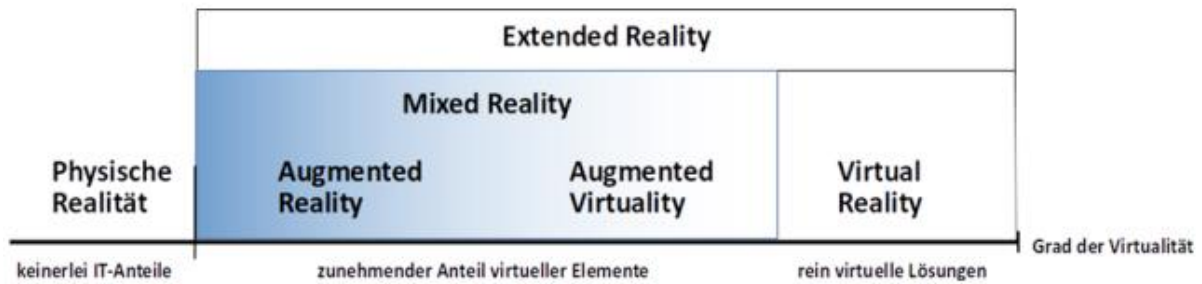


Abbildung 1: AR-/VR-Kontinuum (MATTHIAS KNOLL, 2022)

1.2 Augmented Reality

AR steht für „erweiterte Realität“ und bezeichnet das Einblenden digitaler Inhalte in der realen Welt. Das Ziel ist es, ein Abbild der realen Welt und eine Menge virtueller Elemente so zu überlagern, dass der interagierenden Person durch die Überlagerung in Echtzeit ein Gesamteindruck entsteht. So ist eine direkte Interaktion mit realen und virtuellen Inhalten möglich. Virtuelle Inhalte können verschiedene Formen annehmen, beispielsweise Gebäudeplanungen, historische Informationen, Statusmeldungen, Serviceanweisungen oder Verweise und Abbilder von Zusammenhängen. (MATTHIAS KNOLL, 2022)

Dabei können diese Inhalte je nach Algorithmus beispielsweise auf Oberflächen oder sogenannten Markern platziert werden. Im Alltag tauchen immer mehr Berührungspunkte mit solchen Anwendungen auf, beispielsweise Pokémon GO, IKEA Place App oder Google AR Tiere, die Bedienung ist intuitiv und die technischen Voraussetzungen werden heutzutage bei den meisten Leuten erfüllt. Neben der Darstellung durch mobile Endgeräte sind auch AR-Brillen bzw. Headsets eine Möglichkeit (z.B. Microsoft HoloLens, Meta Quest).

2. A-frame

A-Frame ist ein open-source Web-Framework zur Entwicklung von webbasierten VR- und AR-Erlebnissen. Basierend auf HTML (Hypertext Markup Language) und JavaScript wurde A-Frame von Mozilla entwickelt. Eine der größten Stärken von A-Frame ist die Nutzerfreundlichkeit, wodurch interaktive Szenen für VR und AR mit minimalem Code und ohne tiefgehendes Wissen in WebGL erstellt werden können. Die 3D-Darstellung der Websiteinhalte erfolgt über die WebXR Schnittstelle und die JavaScript-Bibliothek Three.js¹. (MARCOS)

Für die Nutzung von A-Frame bietet Mozilla eine offizielle Dokumentation, in der alle wichtigen Informationen sinnvoll aufgegliedert enthalten sind. Die Nutzercommunity ist vielfältig, ebenfalls hilfreich sind diverse Forenbeiträge oder YouTube-Videos, welche verschiedenste Szenarien und praktische Anwendungsbeispiele behandeln. Neben der Möglichkeit Anwendungen für Augmented Reality zu erstellen, werden auch gängige VR-Headsets wie Vive, Rift, Windows Mixed Reality, Oculus, Meta Quest unterstützt (MARCOS).

2.1 Funktionsweise

Die Verwendung von A-Frame kann über einen Code-Editor im Browser (z.B. Glitch) oder lokal erfolgen, ohne dass eine weitere Installation erforderlich ist. Im Zentrum steht das Entity-Component-System (ECS). Der Ansatz ist simpel, eine Entity ist aus mehreren Components zusammengesetzt. Manche Entities sind durch A-Frame vordefiniert, dazu gehören einfache Primitive, welche häufig zum Einsatz kommen. Eine Entity für eine Box kann zum Beispiel so aussehen:

```
<a-entity geometry="primitive: box; width: 3" material="color: red"></a-entity>
```

Abbildung 2: Quellcode zur Darstellung einer Box, mit der Kantenlänge 3 in rot

¹ Browserübergreifende JavaScript-Bibliothek und Anwendungsprogrammierschnittstelle zur Erstellung und Anzeige animierter 3D-Computergrafiken (Dirksen, 2018)

Tabelle 1: Verschiedene Entitäten zur vereinfachten Darstellung

Elemente	Beschreibung
<a-scene>	Definiert die 3D-Szene
<a-box>	Erstellt einen Würfel
<a-sphere>	Erstellt eine Kugel
<a-cylinder>	Erstellt einen Zylinder
<a-plane>	Erstellt eine flache Ebene
<a-sky>	Erstellt einen Himmel als Hintergrund
<a-camera>	Fügt eine Kamera hinzu (z.B. zur Steuerung der Ansicht)
<a-light>	Fügt eine Lichtquelle hinzu

WebXR ist eine JavaScript-API zum Erstellen immersiver 3D-, VR- und AR-Erlebnisse, bzw. ermöglicht die browserbasierte Darstellung. A-Frame verwendet die WebXR-API, um auf die Sensordaten des VR-Headsets zuzugreifen und die Inhalte zu rendern. Hierbei können verschiedene Plattformen zur Darstellung genutzt werden (Headset, Desktop, Smartphone). Unterstützt werden alle gängigen Browser wie Chrome, Safari, Firefox, Microsoft Edge, Internet Explorer etc. (DÖRNER, 2019)

Eine weitere Möglichkeit ist die Marker-basierte Augmented Reality, dabei wird eine Kamera, zum Beispiel von einem Handy, auf einen Marker gerichtet. Das Objekt wird auf diesem Marker dargestellt, dieser kann in seiner Darstellung individualisiert werden. A-Frame unterstützt AR über die AR.js-Bibliothek, damit werden markerbasierte und markerlose AR ermöglicht. (MARCOS)

2.2 Beispielhafte Erläuterung

Folgend wird kurz die Erstellung einer einfachen A-Frame-Szene anhand eines kurzen Beispiels² gezeigt (Abbildung 3).

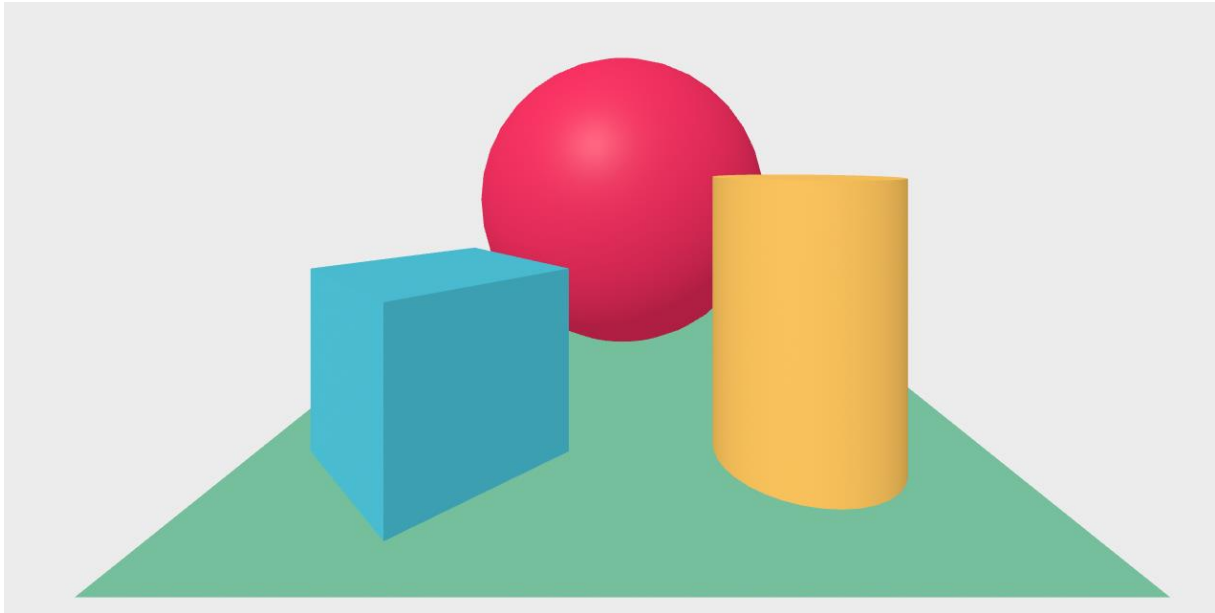


Abbildung 3: Darstellung einer einfachen A-Frame-Szene

Zunächst wird die Grundstruktur der HTML-Datei festgelegt. A-Frame wird von einem CDN (Content Delivery Network) eingebunden, wodurch kein zusätzlicher Download erforderlich ist. Die Szene enthält einfache 3D-Objekte in Form von einem Würfel, einem Zylinder und einer Kugel. Dabei hat jedes Objekt eine Position, Größe, Farbe und eine Rotation. Um die Darstellung in Abbildung 3 zu erhalten ist der Quellcode aus Abbildung 4 erforderlich.

² Beispiel aus den A-Frame Docs (aframe.io/docs/)

```

2  <html>
3  <head>
4  |   <script src="https://aframe.io/releases/1.6.0/aframe.min.js"></script>
5  </head>
6  <body>
7  |   <a-scene>
8  |     <a-box position="-1 0.5 -3" rotation="0 45 0" color="#4CC3D9"></a-box>
9  |     <a-sphere position="0 1.25 -5" radius="1.25" color="#EF2D5E"></a-sphere>
10 |     <a-cylinder position="1 0.75 -3" radius="0.5" height="1.5" color="#FFC65D"></a-cylinder>
11 |     <a-plane position="0 0 -4" rotation="-90 0 0" width="4" height="4" color="#7BC8A4"></a-plane>
12 |     <a-sky color="#ECECEC"></a-sky>
13 |   </a-scene>
14 </body>
15 </html>

```

Abbildung 4: Quellcode zur Erstellung einer einfachen VR-Umgebung mit der Darstellung einfacher Geometrien

Die einfachste Möglichkeit, eine AR-Szene mittels A-Frame zu erstellen zeigt Abbildung 5. Hier wird auf einem vorgegebenen Marker ein roter Würfel abgelegt. Da die Markerdatei als Standard-Datei bereits implementiert ist, muss keine eigene Datei entworfen und hinterlegt werden. Bei der Ausführung der Anwendung wird der Marker mit der Kamera gescannt und erkannt. Anschließend wird das Objekt, in diesem Fall eine rote Box, in der angegebenen Größe und Position auf dem Marker platziert. Die Objektposition wird bei der Bewegung der Kamera eingefroren, so kann man sich das Objekt von allen Seiten betrachten, in dem man in der realen Welt auch den Marker aus der jeweiligen Position scannt. Hierfür muss der Marker dauerhaft im Bild sein.

```

<a-scene embedded arjs>
|   <a-marker preset="hiro">
|     <a-box position="0 0.5 0" color="red"></a-box>
|   </a-marker>
|   <a-entity camera></a-entity>
</a-scene>

```

Abbildung 5: Einfache AR-Szene mittels A-Frame, Darstellung eines roten Würfels auf dem vorgegebenen Hiro-Marker

Anhand beider Beispiele ist deutlich zu erkennen, wie simpel die Anwendung von A-Frame ist. Bereits mit wenigen Codezeilen erhält man eine lauffähige Anwendung, die alle Basisanforderungen erfüllt. Dennoch ist die Flexibilität geboten, je nach Anforderung weitere Funktionen zu ergänzen.

3. Einsatz in der kommunalen Öffentlichkeitsarbeit

Was im ersten Moment wie eine Spielerei klingt, findet heute viele verschiedene Anwendungen, egal ob im privaten oder öffentlichen Sektor. Der Einsatz von VR bietet eine einzigartige Möglichkeit, realitätsnahe Szenarien zu schaffen. Innerhalb einer kommunalen Verwaltung ist der Einsatz von VR und AR im Katastrophenschutz, zu Ausbildungszwecken, zur Risikoeinschätzung aber auch zur Öffentlichkeitsarbeit möglich.

Die Stadt Gelsenkirchen feiert im Sommer 2025 ihr 150-jähriges Stadtjubiläum. Hierfür gibt es diverse Programmpunkte, unter anderem eine Fotoausstellung, Bezirksfeste und zum Abschluss ein großes Stadtfest in der Innenstadt. Das erste Rathaus der Stadt Gelsenkirchen, erbaut 1894, wurde bereits nach 30 Jahren als nicht mehr zeitgemäß angesehen und durch das heutige Hans-Sachs-Haus ersetzt. Nach dem Zusammenschluss der Städte Buer in Westfalen und Gelsenkirchen im Jahr 1928 zog die Verwaltungsspitze ins Rathaus Buer, woraufhin eine Umnutzung des alten Rathauses durch die Polizei stattfand. 1970 erfolgte der Abriss des alten Rathauses. Im Zuge des Jubiläums wurde das alte Rathaus anhand historischer Pläne und Bilddaten rekonstruiert, wodurch ein LoD3³-Gebäudemodell entstanden ist. Für ein Umgebungsmodell wurden in einem kleinen Radius die aktuellen Bestandsgebäude mit der durch ein historisches Luftbild texturierten Geländeoberfläche ergänzt. Der gesamte Prozess wurde mit der Software SketchUp durchgeführt (siehe Abbildung 6).

Im Rahmen der Jubiläumsveranstaltungen soll dieses Modell präsentiert werden, hierfür gibt es unterschiedliche Szenarien, die abgedeckt werden müssen. Einerseits soll dem Bürger die Möglichkeit gegeben werden, das Modell an verschiedenen Orten über ein mobiles Endgerät zu betrachten. Andererseits soll für ausgewählte Termine die Möglichkeit bestehen über ein VR-Headset in die virtuelle Welt eintauchen zu können, um die hautnahe Betrachtung des Modells zu realisieren und dem Nutzer das Gefühl zu geben direkt vor Ort zu sein. Für jedes Szenario gibt es unterschiedliche Lösungsansätze, beispielsweise MindAR⁴ für eine AR-Anwendung oder Unreal Engine⁵ für eine VR-Anwendung. A-Frame hingegen bietet eine Möglichkeit, alle Szenarien im einfachen Rahmen gesammelt abdecken zu können.

³ 3D-Gebäudemodelle werden in LoD1, LoD2 und LoD3 unterteilt, LoD3 stellt das Gebäude als Klötzchenmodell, mit Dachform und texturiert dar (AdV, 2018).

⁴ MindAR ist eine Open-Source-Bibliothek, speziell für AR-Anwendungen.

⁵ Grafik Engine von Epic Games

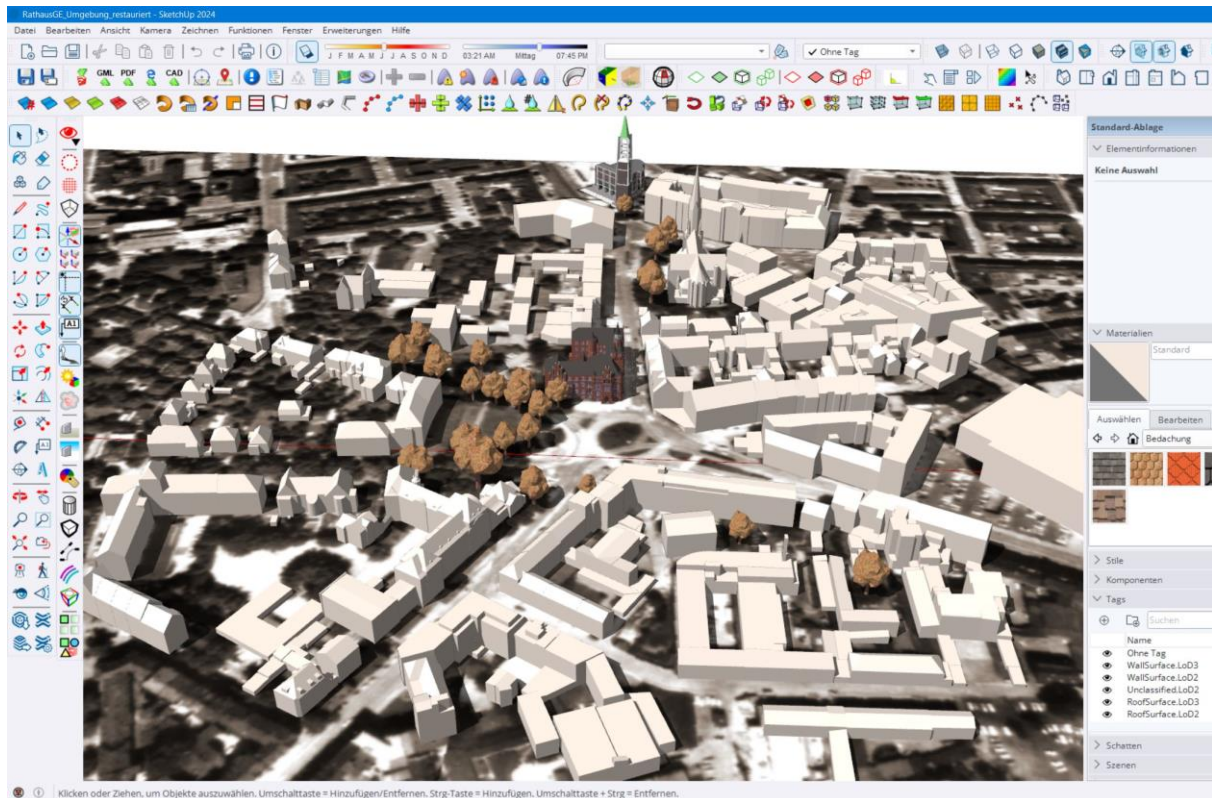


Abbildung 6: Darstellung des erzeugten Umgebungsmodells in SketchUp

3.1 AR

Im Rahmen der Veranstaltungen werden unter Anderem Informationstafeln an verschiedenen Orten aufgestellt. Diese sollen dazu dienen verschiedene Plätze und ihre geschichtliche Entwicklung zu präsentieren. Neben den textlichen Informationen sollen Bilder nicht nur in 2D dargestellt werden, hier kommt nun die AR-Anwendung zum Einsatz.

Über einen QR-Code gelangt der Nutzer auf eine Website, woraufhin auf die Kamera des Endgerätes zugegriffen wird. Nun kann der Marker gescannt werden und das 3D-Modell des alten Rathauses wird auf dem Marker platziert. Hierbei kann ein eigener neuer Marker beispielsweise aus einem Logo entwickelt werden oder der QR-Code fungiert gleichzeitig als Marker (siehe Abbildung 10). Der Marker kann über einen Generator⁶ personalisiert werden. Die HTML-Datei liegt gemeinsam mit der Patt-Datei des Markers und dem gltf-Modell des Rathauses auf dem Web-Server der Stadt Gelsenkirchen. Hierbei ist darauf zu achten, dass eine sichere Verbindung (HTTPS) besteht, damit über die AR.js auf die Kamera

⁶ <https://jeromeetienne.github.io/AR.js/three.js/examples/marker-training/examples/generator.html>

zugreifen zu können. Die Erlaubnis für den Kamerazugriff wird über den Browser direkt beim Nutzer abgefragt. Die Darstellung des Modells auf dem Marker ist in Abbildung 7 zu erkennen.

3.2 VR

Neben der markerbasierten AR-Anwendung ist für Pressetermine und den Internetauftritt eine VR-Lösung gefordert. Hierfür wurde das gesamte Umgebungsmodell aus Abbildung 6 genutzt. Die Steuerung der Oberfläche ist sowohl mit der Maus als auch auf mobilen Endgeräten über die klassischen Touch-Gesten möglich. Die Etablierung der Touch-Steuerung war an dieser Stelle eine größere Herausforderung, hierfür wurden zwei separate .js-Dateien eingebunden, welche jeweils zur Erkennung bzw. zur Umsetzung der Touch-Gesten dienen.

Durch A-Frame werden verschiedene Typen von VR-Headsets unterstützt, hierfür ist der im unteren Bildrand erkennbare „VR-Button“ implementiert. An dieser Stelle wird das städtische Valve Index VR Headset lediglich zur Visualisierung unterstützt, eine Steuerung mithilfe der vorhandenen Controller ist nicht ohne weiteres möglich. Beispielhaft wurde dies mit einer Meta Quest 3 umgesetzt, durch die Unterstützung war eine Steuerung und Bewegung innerhalb des Modells problemlos möglich, hierfür musste lediglich ein einzeiliger Befehl im Quellcode ergänzt werden, dieser ist je nach Headset-Variante individuell.

In diesem Fall lag der Fokus auf einer einfachen Anwendung, ohne weitere Interaktionsmöglichkeiten, da der Nutzungszweck lediglich die Modellbetrachtung war. Für den durchschnittlichen Nutzer sollte eine Bedienung ohne weitere Vorkenntnisse möglich sein.

Das gltf-Umgebungsmodell wurde ebenfalls gemeinsam mit der HTML-Datei und den zugehörigen Steuerungsdateien auf dem Web-Server der Stadt Gelsenkirchen abgelegt.

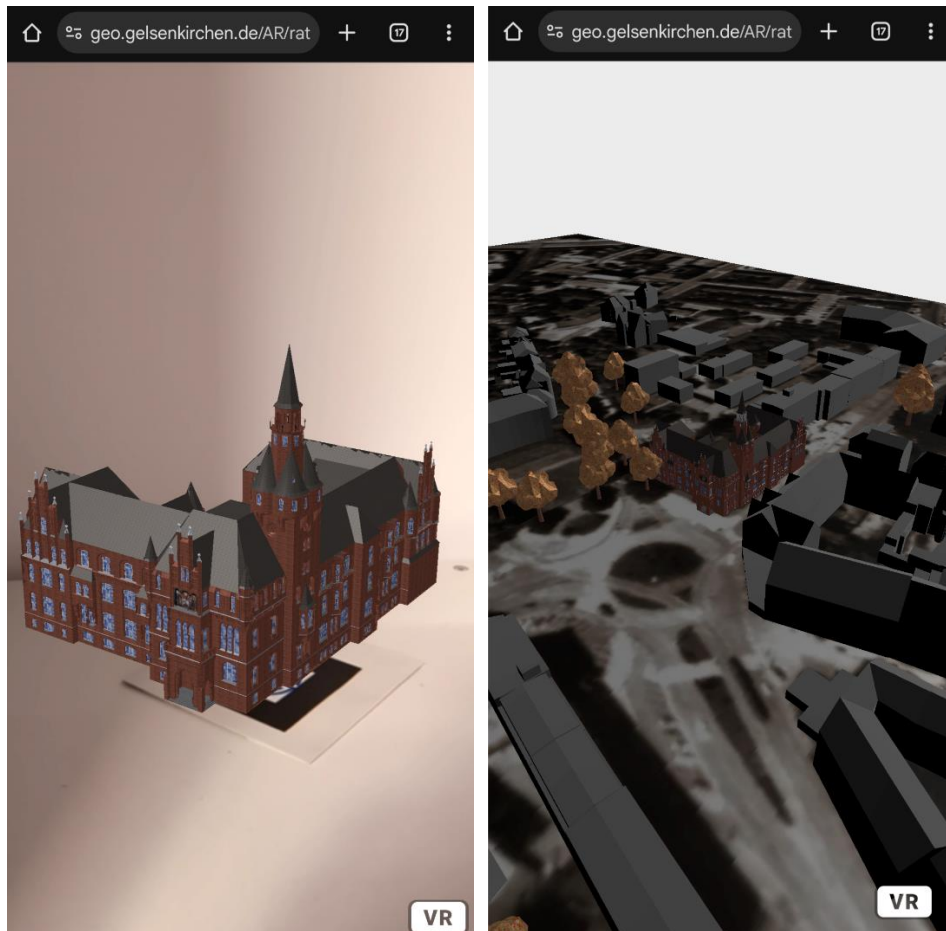


Abbildung 7: Darstellung der AR-Anwendung (links) und VR-Anwendung (rechts) im Browser

4. Ausblick

Die vorliegende Arbeit hat gezeigt, dass A-Frame eine vielseitige und leicht zugängliche Lösung für die Entwicklung von AR- und VR-Anwendungen bietet. Durch die einfache Handhabung und die Nutzung standardisierter Web-Technologien können interaktive und immersive Inhalte mit minimalem Aufwand und ohne besondere Vorkenntnisse erstellt werden. Insbesondere für die kommunale Öffentlichkeitsarbeit eröffnet A-Frame neue Möglichkeiten, historische und städtebauliche Informationen ansprechend zu präsentieren und den Bürgern auf neuem Wege zugänglich zu machen.

Im Rahmen des Stadtjubiläums der Stadt Gelsenkirchen wurde dies anhand eines rekonstruierten Modells des alten Rathauses exemplarisch umgesetzt. Die Kombination aus AR- und VR-Ansätzen ermöglicht es, sowohl mobile Endgeräte als auch VR-Headsets für unterschiedliche Nutzungsszenarien einzubinden. Herausforderungen wie die Implementierung der Touch-Steuerungen oder die Sicherstellung einer stabilen WebXR-Integration konnten erfolgreich gelöst werden.

Ein großer Nachteil ist die fehlende Unterstützung des städtischen VR-Headsets, an dieser Stelle findet A-Frame keine Anwendung. Eine mögliche Alternative zur Visualisierung mittels VR-Brille kann eine Game Engine (Unreal Engine⁷) oder das SketchUp-VR Plugin sein. Im Rahmen der Untersuchungen wurde als Alternative für die AR-Nutzung MindAR getestet, auch hier bietet sich durch die Spezialisierung auf AR-Anwendung eine leistungsstarke Alternative. Hier liegen die Vorteile besonders im fortschrittlichen Tracking der Marker für eine präzisere Platzierung.

A-Frame ist eine gute All-in-One Lösung für einfache AR- und VR-Anwendungen mit vielen verschiedenen Anwendungsbereichen, auch innerhalb einer Stadtverwaltung. Es eignet sich gut für kleinere Anwendungen und Prototypen. Durch die limitierte Performance kann es bei komplexeren und ressourcenintensiveren Anwendungen schnell zu Leistungseinbrüchen kommen, was zu Latenzproblemen oder geringeren Frameraten führt. Für spezifische Geräte und Plattformen müssen zusätzliche Anpassungen durchgeführt oder Alternativen gesucht werden.

Besonders in kommunalen Verwaltungen steckt die Verwendung von AR und VR noch in den Anfängen, doch auch dort ist den kommenden Jahren mit einem höheren Anwendungsaufkommen zu rechnen. Hierbei bewirken nicht nur in SketchUp modellierte LoD3-Modell eine Veränderung, auch die Echtzeit-Visualisierung von Punktwolken und bildbasierter Modellierungen (beispielweise durch Laserscanning, Photogrammetrie und kombiniertem Gaussian-Splatting⁸) werden sich in den nächsten Jahren etablieren. A-Frame bietet zum jetzigen Zeitpunkt einen perfekten Einstieg in die Visualisierung sowohl im AR- als auch im VR-Bereich und ist besonders im Verwaltungsbereich ein erster Schritt in die Richtung der 3D-Visualisierung.

⁷ Entwickelt von Epic Games

⁸ Fotorealistische Darstellung von 3D-Szenen auf der Basis von Punktwolken und Bilddaten

5. Quellcode

5.1 AR

```
1  <!-- Grundlegendes HTML-Dokument mit AR.js für Augmented Reality -->
2  <html>
3
4  <head>
5      <title>ar-demo</title>
6      <!-- Meta-Tag für die Beschreibung der Seite -->
7      <meta name="description" content="ar-demo">
8
9      <!-- Einbindung der A-Frame-Bibliothek für WebXR -->
10     <script src="https://aframe.io/releases/1.6.0/aframe.min.js"></script>
11
12     <!-- Einbindung von AR.js für Marker-basierte AR-Erfahrungen -->
13     <script src="https://raw.githack.com/AR-js-org/AR.js/master/aframe/build/aframe-ar.js"></script>
14 </head>
15
16 <body style='margin : 0px; overflow: hidden;'>
17     <!-- A-Frame Szene für Augmented Reality -->
18     <a-scene embedded arjs='sourceType: webcam;'
19         gltf-model="
20             dracoDecoderPath: https://www.gstatic.com/draco/versioned/decoders/1.5.6/;
21             meshoptDecoderPath: https://unpkg.com/meshoptimizer@0.19.0/meshopt_decoder.js;"
22
23     <!-- AR-Marker: Verwendet ein Musterbild als Tracking-Marker -->
24     <a-marker-camera type='pattern' url='assets/GELogo.patt'>
25         <!-- 3D-Modell wird auf dem Marker platziert -->
26         <a-entity gltf-model="assets/opt_RathausGE_NullpunktRathaus_solo.glb"
27             scale="0.05 0.05 0.05"
28             position="0 0.5 0"></a-entity>
29     </a-marker-camera>
30
31     <!-- Standard-Kamera in der Szene -->
32     <a-entity camera></a-entity>
33 </a-scene>
34 </body>
35
36 </html>
```

Abbildung 8: Quellcode AR-Anwendung

5.2 VR

```
1  <!DOCTYPE html>
2  <html lang="de">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Rathaus_GE</title>
8      <!-- Einbinden von A-Frame für die Erstellung der 3D-Ansicht -->
9      <script src="https://aframe.io/releases/1.6.0/aframe.min.js"></script>
10     <!-- Einbinden der Touchsteuerung -->
11     <script src="./gesture-detector.js"></script>
12     <script src="./gesture-handler.js"></script>
13 </head>
```



```

15 <body>
16 <!-- A-Frame Szene, die das 3D-Modell darstellt -->
17 <a-scene gesture-detector gltf-model="
18
19 dracoDecoderPath: https://www.gstatic.com/draco/versioned/decoders/1.5.6/;
20 meshoptDecoderPath: https://unpkg.com/meshoptimizer@0.19.0/meshopt\_decoder.js";
21 <!-- Hintergrund (Himmel) der Szene -->
22 <a-sky color="#E0E0E0"></a-sky>
23
24 <!-- Lichtquellen für bessere Beleuchtung -->
25 <a-light type="ambient" color="#ffffff" intensity="0.5"></a-light>
26 <a-light type="directional" position="1 1 0" intensity="0.8"></a-light>
27
28 <!-- Kamera, die die Szene anzeigt -->
29 <a-camera id="camera" look-controls="enabled: false" position="0 20 60"
30   mouse-controls="minDistance: 0.1; maxDistance: 5000; zoomSpeed: 0.1"></a-camera>
31
32 <!-- 3D Modell (das .glb File wird hier eingebunden) -->
33 <a-entity id="outer">
34   <a-entity id="inner" gesture-handler="minScale: 0.25; maxScale:10">
35     <a-entity id="model" gltf-model="assets/opt_RathausGE_Umgebung_NullpunktRathaus.glb"
36       position="0 0 0"
37       scale="1 1 1">
38     </a-entity>
39   </a-entity>
40 </a-entity>
41 </a-scene>
42
43 <!-- Steuerungs-Logik -->
44 <script>
45   // Definiert eine benutzerdefinierte A-Frame-Komponente für Maussteuerung
46   AFRAME.registerComponent('mouse-controls', {
47     schema: {
48       minDistance: { type: 'number', default: 0.1 },
49       maxDistance: { type: 'number', default: 5000 },
50       zoomSpeed: { type: 'number', default: 0.1 },
51       rotateSpeed: { type: 'number', default: 0.005 },
52       panSpeed: { type: 'number', default: 0.1 }
53     },
54     init: function () {
55       const camera = this.el.getObject3D('camera');
56       const model = document.querySelector('#model').object3D;
57       let isRotating = false, isPanning = false, prevPos = {x: 0, y: 0};
58
59       // Zoom Funktion
60       window.addEventListener('wheel', (e) => {
61         const zoom = e.deltaY * this.data.zoomSpeed;
62         camera.position.z = THREE.MathUtils.clamp(camera.position.z + zoom, this.data.minDistance, this.data.maxDistance);
63         //model.scale = new THREE.Vector3(model.scale.x + zoom, model.scale.y + zoom, model.scale.z + zoom);
64       });
65
66       // Mausinteraktionen
67       window.addEventListener('mousedown', (e) => {
68         isRotating = e.button === 0; // Linke Maustaste = Rotation
69         isPanning = e.button === 2; // Rechte Maustaste = Verschieben
70         prevPos = {x: e.clientX, y: e.clientY};
71       });
72       window.addEventListener('mouseup', () => {
73         isRotating = isPanning = false;
74       });
75
76       // Mausbewegungen
77       window.addEventListener('mousemove', (e) => {
78         const dx = e.clientX - prevPos.x;
79         const dy = e.clientY - prevPos.y;
80         prevPos = {x: e.clientX, y: e.clientY};
81         if (isRotating) {
82           model.rotation.y += dx * this.data.rotateSpeed; // Rotation
83         }
84         if (isPanning) {
85           const panX = -dx * this.data.panSpeed, panY = dy * this.data.panSpeed;
86           const cameraRight = new THREE.Vector3().crossVectors(camera.up, camera.getWorldDirection(new THREE.Vector3())).normalize();
87           model.position.addScaledVector(cameraRight, panX).addScaledVector(camera.up, panY); // Verschieben
88         }
89       });
90     }
91   });
92 </script>
93 </body>
94 </html>

```

Abbildung 9: Quellcode VR-Anwendung

6. Marker und QR-Code-Abbildungen



Abbildung 10: QR-Code AR (links), QR-Code VR (mitte), Marker AR (rechts)

6.1 URL Ar-Anwendung

https://geo.gelsenkirchen.de/AR/rathaus/AR_index.html

6.2 URL VR-Anwendung

https://geo.gelsenkirchen.de/AR/rathaus/VR_index.html

Literaturverzeichnis

AdV. (21. 03 2018). Dokumentation zur Modellierung der Geoinformation des amtlichen Vermessungswesens (GeoInfoDok).

Dirksen, J. (2018). *Learn Three.js*. Packt Publishing.

Dörner, B. G. (2019). *Virtual und Augmented Reality (AR/VR)*. Springer.

Langer, E. (2020). *Medieninnovation AR und VR*. Springer.

Marcos, D. (kein Datum). *A-Frame-Docs*. Von <https://aframe.io/docs/1.7.0> abgerufen

Matthias Knoll, S. S. (2022). *Augmented Reality und Virtual Reality - Einsatz im Kontext von Arbeit , Forschung und Lehre*. Springer.